

Module 01 — The Platform

FOR DUMMIES

Goal: a Debian 13 VM in Aston's vCenter running Docker, Containerlab, and the Python automation toolchain, with the cEOS image imported.

This is the only module where `setup.sh` does the driving — it's a platform build, and there's nothing educational about watching `apt` scroll. The networking education starts in module 02.

1. Build the VM (one time)

Follow [docs/vcenter-vm-setup.md](#) — about 10 minutes of clicking in the vSphere Client. The three settings people get wrong, so you don't:

- Network adapter: VMXNET3, attached to the routable port group (ask whoever owns the resource pool which one — it's the /23 with DHCP that's reachable from Aston WiFi and the VPN). *Not* an isolated lab network: that's the old GNS3 jump-host misery this whole project exists to escape.
- Guest OS: Other 4.x or later Linux (64-bit)
- Install from the DVD-1 ISO, not netinst. Netinst needs internet *during* the install; if the network isn't right yet, you get a crippled system missing `sudo`, `ifupdown`, and SSH. (Home lab on Proxmox instead? [docs/proxmox-vm-setup.md](#) — and use a VM, not an LXC; the doc explains why.)

At the Debian installer's software selection: no desktop, yes SSH server, yes standard system utilities.

2. Get this repo onto the VM

Options compared in [docs/distributing.md](#). If the team repo already exists:

```
sudo apt install -y git
git clone <aston-internal-git-url> ~/clab-bootstrap
cd ~/clab-bootstrap
```

3. Run the guided setup

```
./setup.sh
```

Answer the prompts. Recommendations for this course: yes to gum (nicer prompts, and it's a tool worth knowing), uv for the Python environment, and when it asks about the cEOS image, follow its instructions. Fastest route is the [team Drive](#) → ContainerLab/Arista Images/ — take the `.tar.xz` and its `.sha512sum`. Otherwise download it from Arista's portal on your laptop (needs your own free Arista

account), verify the sha512, and `scp` it to the VM. The script re-verifies the hash on the VM side before importing; a corrupted image produces genuinely baffling boot failures later, which is why it's this paranoid.

Say yes to deploying the test topology at the end — module 02 uses it. Skip the secrets prompt for now; that's module 06.

Then log out and back in — your user just joined the `docker` and `clab_admins` groups, and group changes only apply to new sessions.

4. Checkpoint

All four of these must work before module 02:

```
docker run hello-world           # docker works, without sudo
containerlab version             # containerlab installed
docker images | grep ceos       # cEOS image imported
netauto && python3 -c "import nornir, napalm, netmiko; print('automation env OK')"
```

If any fail: `./setup.sh` again (it skips finished steps), and the full log is at `/tmp/clab-setup.log`. The README's troubleshooting table covers the common ones — no DHCP lease, docker apt 404, permission denied on the scripts.

What you learned

Honestly? Not much yet — that's the point. The platform build is automated *because* it's not the interesting part. What you should take away is where the seams are: a hypervisor VM, a container runtime inside it, a lab orchestrator on top of that, and a Python environment alongside. Every later module lives in the top two layers.

Next: [Module 02 — Containerlab basics](#)